

IGMP Snooping no POX

Universidade de Caxias do Sul - Redes de Computadores

Prof. Maria de Fátima

Gabriel Vieira e Rafael Graunke

1. Descrição da Funcionalidade Implementada

Objetivo, Motivação e Benefícios

O POX é um controlador SDN em Python que suporta OpenFlow 1.0. Em redes SDN, o controlador decide como o tráfego é encaminhado pelos switches. Por padrão, quando um pacote multicast chega a um switch gerenciado pelo POX, o controlador instala regras de encaminhamento que replicam o tráfego para todas as portas, comportamento equivalente a um broadcast. Isso ocorre porque o POX não possui nenhuma lógica para distinguir quais hosts têm interesse em receber determinado grupo multicast.

O IGMP (*Internet Group Management Protocol*) é o protocolo utilizado por hosts para informar à rede que desejam receber tráfego de um grupo multicast específico. Um host envia uma mensagem *Membership Report* (tipo `0x16`) ao ingressar num grupo e uma mensagem *Leave Group* (tipo `0x17`) ao sair. Sem que o controlador interprete essas mensagens, todo tráfego multicast é tratado como broadcast, desperdiçando banda e processamento nos hosts que não têm interesse no conteúdo.

Este trabalho implementa **IGMP Snooping** no POX: o controlador passa a interceptar e interpretar os cabeçalhos IGMP, mantendo uma tabela de quais portas do switch possuem hosts inscritos em cada grupo multicast. Com base nessa tabela, instala regras OpenFlow direcionadas, de modo que o tráfego multicast chegue apenas às portas com assinantes ativos.

Benefícios:

- Elimina flooding desnecessário de tráfego multicast
- Reduz processamento em hosts não inscritos
- Funcionalidade genuinamente ausente no POX, integração puramente aditiva

Requisitos Atendidos

- Interpretação dos campos `type` e `group address` do cabeçalho IGMP
- Manutenção de tabela `(switch, grupo) -> conjunto de portas`
- Instalação de regras OpenFlow direcionadas ao receber tráfego multicast
- Remoção de porta da tabela ao receber *Leave Group*
- Fallback para flood quando grupo não possui assinantes conhecidos

Limitações

- Suporta apenas IGMPv2 (tipos `0x16` e `0x17`); IGMPv3 não implementado; hosts Mininet configurados com `force_igmp_version=2` para compatibilidade
- Sem suporte a *Querier* IGMP (o controlador não envia *General Queries*)

- Entradas na tabela não expiram por tempo (*timeout*); remoção ocorre apenas via *Leave*
- Testado exclusivamente com Open vSwitch em topologia Mininet

2. Integração com o Software Original

Arquivos Adicionados

Arquivo	Papel
<code>pox/forwarding/igmp_snooping.py</code>	Componente POX: interpreta cabeçalhos IGMP e instala regras OpenFlow
<code>ext/topo_igmp_demo.py</code>	Topologia Mininet para demonstração (3 hosts + 1 switch)

Arquivos Originais Modificados

Nenhum arquivo original do POX foi modificado. A integração é puramente aditiva: o novo componente se registra nos eventos do core do POX sem alterar nenhum módulo existente.

Ponto de Integração

O POX expõe um sistema de eventos ao qual qualquer componente pode se registrar. O componente `igmp_snooping` utiliza dois eventos já existentes no core:

Evento POX	Quando dispara	Uso no componente
<code>ConnectionUp</code>	Switch conecta ao controlador	Inicializa estrutura de dados para o switch
<code>PacketIn</code>	Switch envia pacote ao controlador	Inspeciona cabeçalho IGMP; atualiza tabela; instala regra

Não é necessário modificar o core do POX nem os módulos de encaminhamento existentes (`l2_learning` , `l3_learning` , etc.). O componente coexiste com eles e pode ser carregado de forma independente.

3. Detalhamento Técnico da Implementação

Descrição do Desenvolvimento

O componente foi desenvolvido como um módulo autônomo em `pox/forwarding/igmp_snooping.py` , sem modificar nenhum arquivo existente do POX. A decisão central foi implementar o parser IGMP manualmente via `struct.unpack` , em vez de utilizar o parser já existente em `pox/lib/packet/igmp.py` , tornando explícita a interpretação de cada campo do cabeçalho.

O estado do componente é mantido em um dicionário aninhado (`group_table`) indexado por DPID do switch e endereço do grupo multicast. Cada entrada contém o conjunto de portas com assinantes ativos.

A integração com o POX se dá exclusivamente pelo sistema de eventos do core: `ConnectionUp` para inicializar o estado por switch, e `PacketIn` para processar pacotes IGMP e encaminhar tráfego multicast.

pox/forwarding/igmp_snooping.py

Símbolo	Tipo	Papel
<code>parse_igmp(raw)</code>	função	deserializa manualmente os 8 bytes do cabeçalho IGMPv2 via <code>struct.unpack("!BBH4s", ...)</code>
<code>IGMPSnooping</code>	classe	componente POX principal; mantém <code>group_table[dpid][group] = set(ports)</code>
<code>__init__()</code>	método	inicializa <code>group_table</code> e registra handlers no <code>core.openflow</code>
<code>_handle_ConnectionUp(event)</code>	método	inicializa entrada <code>group_table[dpid] = {}</code> quando switch conecta
<code>_handle_PacketIn(event)</code>	método	identifica pacotes IGMP (IPv4 proto=2) e tráfego multicast; delega ao método adequado
<code>_process_igmp(event, igmp)</code>	método	lê <code>igmp['type']</code> e <code>igmp['group']</code> ; atualiza tabela; aciona instalação de regra
<code>_install_multicast_flow(event, group)</code>	método	instala regra OpenFlow com <code>nw_dst=group</code> , output para cada porta inscrita
<code>_install_drop_flow(event, group)</code>	método	instala regra de descarte quando grupo não tem mais assinantes
<code>_forward_multicast(event, ipv4_pkt)</code>	método	encaminha tráfego multicast não-IGMP para portas conhecidas, ou flood
<code>_flood(event)</code>	método	envia <code>packet_out</code> com <code>OFPP_FLOOD</code>
<code>launch()</code>	função	entry point para <code>pox.py</code> ; registra instância via <code>core.registerNew</code>

Fluxo de Execução

- `pox.py` carrega `igmp_snooping`
- `IGMPSnooping.__init__()` registra os handlers `_handle_ConnectionUp` e `_handle_PacketIn`
- Ao receber `ConnectionUp`: inicializa `group_table[dpid] = {}`
- Ao receber `PacketIn`: extrai o payload e tenta parsear como IGMP
 - Se não for IGMP: encaminha normalmente (flood)
 - Se for IGMP: chama `_process_igmp(dpid, port, igmp)`
 - Se `type == 0x16` (Join): adiciona a porta ao grupo na tabela
 - Se `type == 0x17` (Leave): remove a porta do grupo na tabela
 - Chama `_install_multicast_flow()`: instala regra com destino igual ao grupo e saída apenas nas portas inscritas

Interpretação de Cabeçalhos de Protocolos

Cabeçalho IGMP (8 bytes)

Campo	Offset	Tamanho	Valores interpretados
Type	0	1 byte	0x16 = Membership Report (Join); 0x17 = Leave Group
Max Resp Time	1	1 byte	ignorado nesta implementação
Checksum	2	2 bytes	integridade do cabeçalho (big-endian)
Group Address	4	4 bytes	endereço IPv4 multicast do grupo (ex.: 224.1.1.1)

O cabeçalho é deserializado manualmente via `struct.unpack`, sem uso do parser existente em `pox/lib/packet/igmp.py`:

```
import struct
import socket

def parse_igmp(raw):
    type_, max_resp, checksum, group_bytes = struct.unpack("!BBH4s", raw[:8])
    return {
        'type': type_,
        'max_resp': max_resp,
        'checksum': checksum,
        'group': socket.inet_ntoa(group_bytes),
    }
```

`struct.unpack("!BBH4s", ...)` mapeia diretamente os bytes do wire:

- ! : big-endian (network byte order)
- B : 1 byte unsigned: `type`
- B : 1 byte unsigned: `max_resp_time`
- H : 2 bytes unsigned: `checksum`
- 4s : 4 bytes raw, convertido para string IPv4 com `socket.inet_ntoa`

Encapsulamento

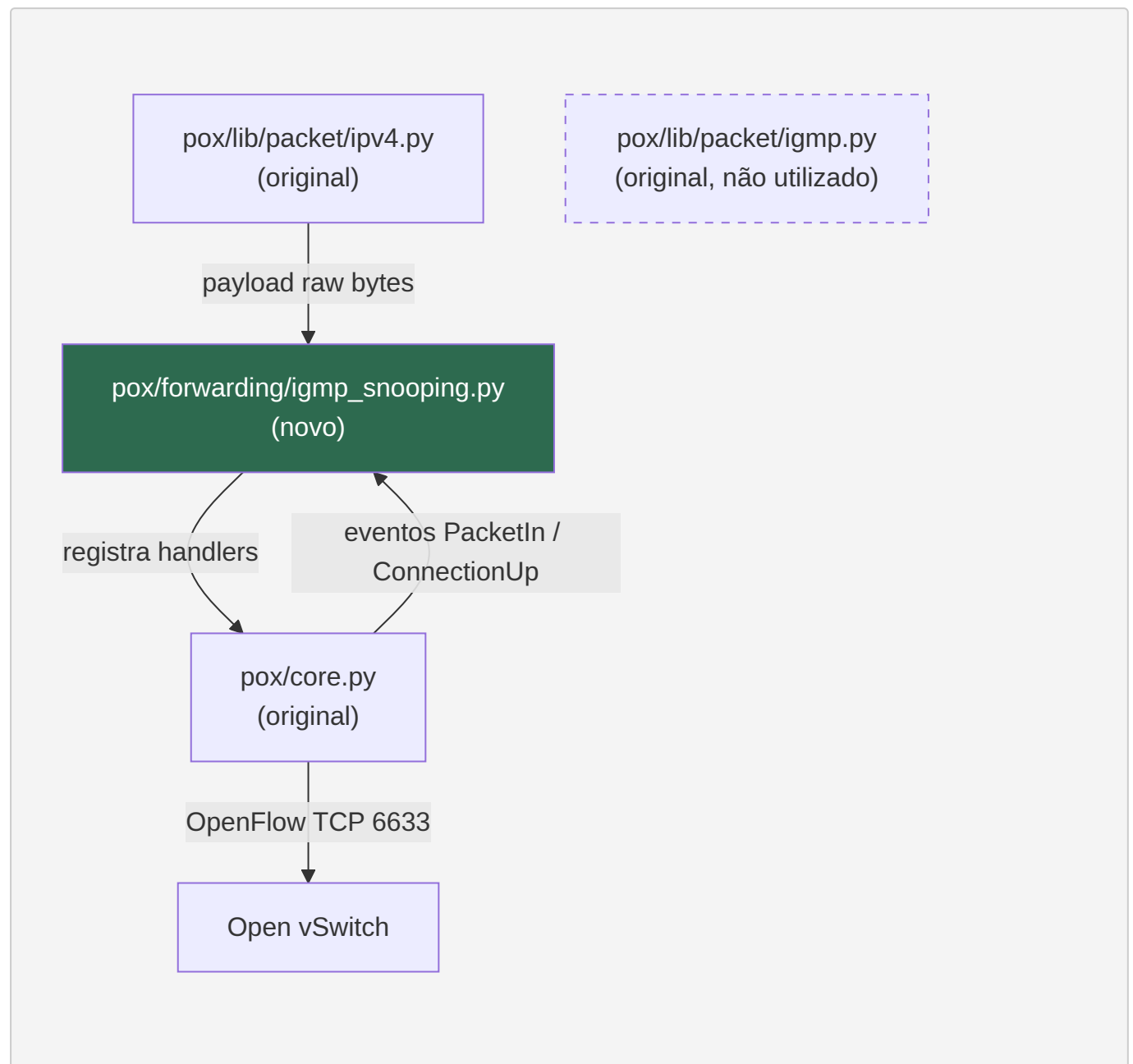
O pacote IGMP chega encapsulado da seguinte forma:

- Camada 2: Ethernet com destino `01:00:5e:xx:xx:xx` (endereço multicast mapeado)
- Camada 3: IPv4 com protocolo `2` (reservado para IGMP) e destino `224.x.x.x`
- Camada 4: IGMP com `type` igual a `0x16` ou `0x17` e `group` com o endereço do grupo

O protocolo IP reserva o número `2` para IGMP no campo `protocol` do cabeçalho IPv4. O endereço de destino multicast no cabeçalho Ethernet é derivado do grupo (`01:00:5e` + 23 bits do endereço IP).

4. Diagramas

Diagrama de Módulos



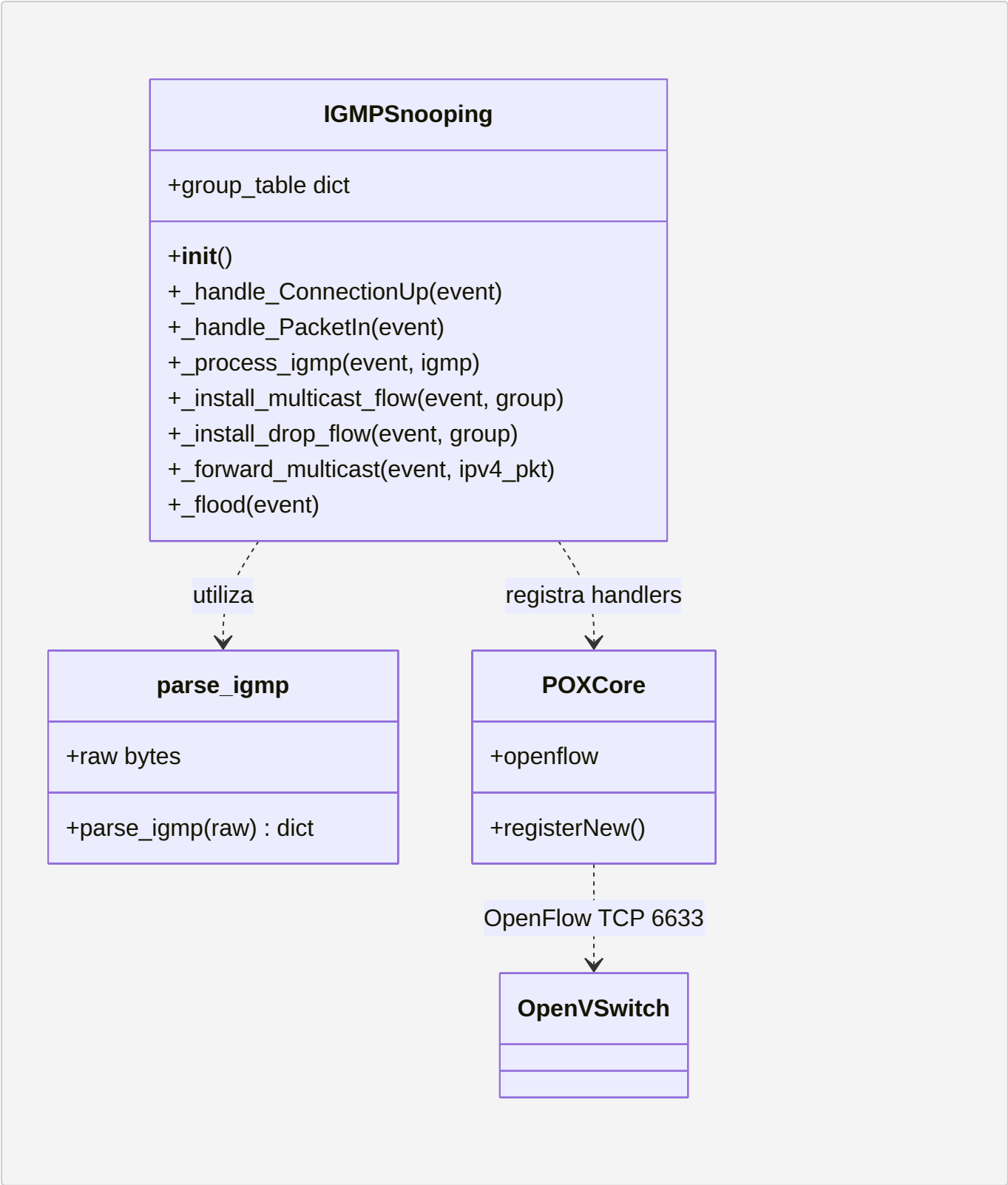
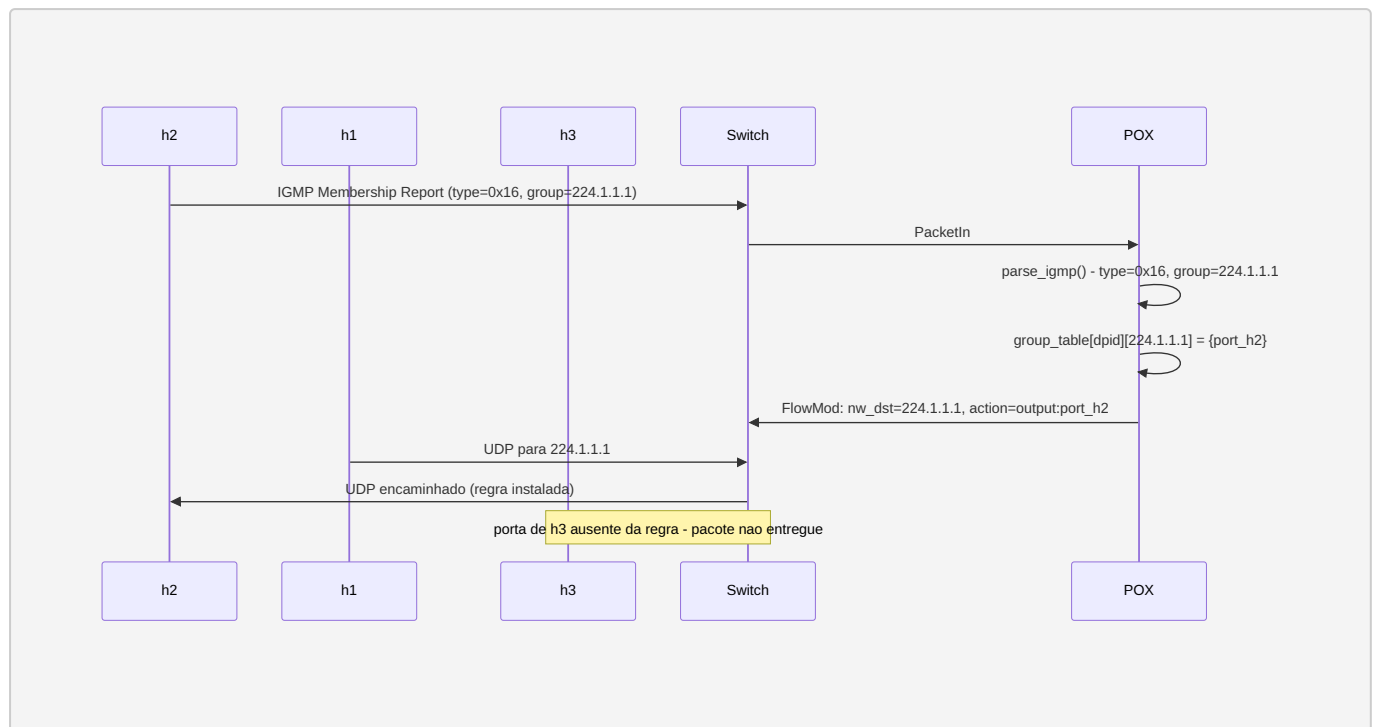


Diagrama de Sequência



5. Procedimentos de Execução e Validação

Requisitos

```
python3 --version      # Python 3.x
ovs-vsctl --version    # Open vSwitch
mn --version           # Mininet
iperf --version         # iperf (gerador de tráfego UDP)
tcpdump --version      # captura de pacotes
```

Demonstração: Ausência de Snooping (baseline)

Prova que sem o componente, h3 recebe tráfego multicast sem ter se inscrito.

Terminal 1: POX sem snooping:

```
cd ~/projects/pox
python3 pox.py log.level --DEBUG forwarding.l2_learning
```

Terminal 2: Mininet:

```
sudo python3 ext/topo_igmp_demo.py
```

No prompt **mininet**> :

```
mininet> h2 iperf -s -u -B 224.1.1.1 -i 1 &  
mininet> h2 tcpdump -i h2-eth0 udp and dst 224.1.1.1 -v &  
mininet> h3 tcpdump -i h3-eth0 udp and dst 224.1.1.1 -v &  
mininet> h1 iperf -c 224.1.1.1 -u --ttl 5 -t 10  
mininet> h2 kill %2  
mininet> h3 kill %1
```

Resultado esperado: h3 captura pacotes UDP, flooding sem controle.

Demonstração: Com IGMP Snooping

Terminal 1: POX com snooping:

```
cd ~/projects/pox  
python3 pox.py log.level --DEBUG forwarding.igmp_snooping
```

Terminal 2: Mininet (mesmo script):

```
sudo python3 ext/topo_igmp_demo.py
```

No prompt **mininet**> :


```
mininet> h2 iperf -s -u -B 224.1.1.1 -i 1 &
mininet> h2 tcpdump -i h2-eth0 udp and dst 224.1.1.1 -v &
mininet> h3 tcpdump -i h3-eth0 udp and dst 224.1.1.1 -v &
mininet> h1 iperf -c 224.1.1.1 -u --ttl 5 -t 10
mininet> h2 kill %2
mininet> h3 kill %1
```

Resultado esperado: h3 não captura nada, tráfego direcionado apenas a h2.

Casos de Teste

#	Caso	Entrada	Resultado Esperado
T1	Flood sem snooping	<code>12_learning</code> + multicast h1 para h2	h3 captura pacotes UDP
T2	Sem inscrição	<code>igmp_snooping</code> + multicast sem Join	tráfego não chega a nenhum host
T3	Join de h2	h2 executa <code>iperf -s -B 224.1.1.1</code>	h2 recebe tráfego; h3 não recebe
T4	Leave de h2	h2 encerra iperf (envia Leave)	h2 para de receber tráfego
T5	Join de h2 e h3	ambos executam iperf -s	ambos recebem tráfego

Evidências

T1 — Sem IGMP Snooping (`12_learning`): h3 recebe tráfego multicast sem estar inscrito

```

*** Starting CLI:
mininet> h2 iperf -s -u -B 224.1.1.1 -i 1 &
mininet> h2 tcpdump -i h2-eth0 udp and dst 224.1.1.1 -v &
-----
Server listening on UDP port 5001
Joining multicast group 224.1.1.1
Server set to single client traffic mode (per multicast receive)
UDP buffer size: 208 KByte (default)
-----
mininet> h3 tcpdump -i h3-eth0 udp and dst 224.1.1.1 -v &
mininet> h1 iperf -c 224.1.1.1 -u --ttl 5 -t 10
-----
Client connecting to 224.1.1.1, UDP port 5001
Sending 1470 byte datagrams, IPG target: 11215.21 us (kalman adjust)
UDP buffer size: 208 KByte (default)
-----
[ 1] local 10.0.0.1 port 51066 connected with 224.1.1.1 port 5001
^C[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-0.4600 sec 61.7 KBytes  1.10 Mbits/sec
[ 1] Sent 44 datagrams
-----
mininet> h2 kill %2
tcpdump: listening on h2-eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
[ 1] local 224.1.1.1 port 5001 connected with 10.0.0.1 port 51066
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagrams
[ 1] 0.0000-0.5001 sec 61.7 KBytes  1.01 Mbits/sec  2.077 ms  0/43 (0%)
[ 1] 0.0000-0.5001 sec 61.7 KBytes  1.01 Mbits/sec  0.383 ms  0/43 (0%)
13:50:13.120253 IP (tos 0x0, ttl 5, id 60097, offset 0, flags [DF], proto UDP (17), length 1498)
    10.0.0.1.51066 > 224.1.1.1.5001: UDP, length 1470
13:50:13.132076 IP (tos 0x0, ttl 5, id 60098, offset 0, flags [DF], proto UDP (17), length 1498)
    10.0.0.1.51066 > 224.1.1.1.5001: UDP, length 1470
13:50:13.142061 IP (tos 0x0, ttl 5, id 60099, offset 0, flags [DF], proto UDP (17), length 1498)
    10.0.0.1.51066 > 224.1.1.1.5001: UDP, length 1470
13:50:13.153443 IP (tos 0x0, ttl 5, id 60100, offset 0, flags [DF], proto UDP (17), length 1498)
    10.0.0.1.51066 > 224.1.1.1.5001: UDP, length 1470
13:50:13.164556 IP (tos 0x0, ttl 5, id 60101, offset 0, flags [DF], proto UDP (17), length 1498)
    10.0.0.1.51066 > 224.1.1.1.5001: UDP, length 1470
13:50:13.175633 IP (tos 0x0, ttl 5, id 60102, offset 0, flags [DF], proto UDP (17), length 1498)
    10.0.0.1.51066 > 224.1.1.1.5001: UDP, length 1470
13:50:13.187154mininet>
-----
mininet> h3 kill %1
tcpdump: listening on h3-eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
13:50:13.120257 IP (tos 0x0, ttl 5, id 60097, offset 0, flags [DF], proto UDP (17), length 1498)
    10.0.0.1.51066 > 224.1.1.1.5001: UDP, length 1470
13:50:13.132080 IP (tos 0x0, ttl 5, id 60098, offset 0, flags [DF], proto UDP (17), length 1498)
    10.0.0.1.51066 > 224.1.1.1.5001: UDP, length 1470
13:50:13.142063 IP (tos 0x0, ttl 5, id 60099, offset 0, flags [DF], proto UDP (17), length 1498)
    10.0.0.1.51066 > 224.1.1.1.5001: UDP, length 1470
13:50:13.153446 IP (tos 0x0, ttl 5, id 60100, offset 0, flags [DF], proto UDP (17), length 1498)
    10.0.0.1.51066 > 224.1.1.1.5001: UDP, length 1470
13:50:13.164560 IP (tos 0x0, ttl 5, id 60101, offset 0, flags [DF], proto UDP (17), length 1498)
    10.0.0.1.51066 > 224.1.1.1.5001: UDP, length 1470
13:50:13.175633 IP (tos 0x0, ttl 5, id 60102, offset 0, flags [DF], proto UDP (17), length 1498)
    10.0.0.1.51066 > 224.1.1.1.5001: UDP, length 1470
13:50:13.187154mininet>

```

T3 — Com IGMP Snooping: h3 captura 0 pacotes; h2 recebe normalmente

```

*** Starting CLI:
mininet> h2 iperf -s -u -B 224.1.1.1 -i 1 &
mininet> h2 tcpdump -i h2-eth0 udp and dst 224.1.1.1 -v &
-----
Server listening on UDP port 5001
Joining multicast group 224.1.1.1
Server set to single client traffic mode (per multicast receive)
UDP buffer size: 208 KByte (default)
-----
mininet> h3 tcpdump -i h3-eth0 udp and dst 224.1.1.1 -v &
mininet> h1 iperf -c 224.1.1.1 -u --ttl 5 -t 10
-----
Client connecting to 224.1.1.1, UDP port 5001
Sending 1470 byte datagrams, IPG target: 11215.21 us (kalman adjust)
UDP buffer size: 208 KByte (default)
-----
[ 1] local 10.0.0.1 port 42193 connected with 224.1.1.1 port 5001
^C[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-0.5274 sec 70.3 KBytes  1.09 Mb/s/sec
[ 1] Sent 50 datagrams
mininet> h3 kill %1
tcpdump: listening on h3-eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
0 packets captured
0 packets received by filter
0 packets dropped by kernel
mininet> h2 kill %2
tcpdump: listening on h2-eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
[ 1] local 224.1.1.1 port 5001 connected with 10.0.0.1 port 42193
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagrams
[ 1] 0.0000-0.5271 sec 70.3 KBytes  1.09 Mb/s/sec  0.017 ms  0/49 (0%)
[ 1] 0.0000-0.5271 sec 70.3 KBytes  1.09 Mb/s/sec  0.017 ms  0/49 (0%)
13:57:55.949186 IP (tos 0x0, ttl 5, id 3962, offset 0, flags [DF], proto UDP (17), length 1498)
10.0.0.1.42193 > 224.1.1.1.5001: UDP, length 1470
13:57:55.960268 IP (tos 0x0, ttl 5, id 3963, offset 0, flags [DF], proto UDP (17), length 1498)
10.0.0.1.42193 > 224.1.1.1.5001: UDP, length 1470
13:57:55.960313 IP (tos 0x0, ttl 5, id 3964, offset 0, flags [DF], proto UDP (17), length 1498)
10.0.0.1.42193 > 224.1.1.1.5001: UDP, length 1470
13:57:55.971620 IP (tos 0x0, ttl 5, id 3965, offset 0, flags [DF], proto UDP (17), length 1498)
10.0.0.1.42193 > 224.1.1.1.5001: UDP, length 1470
13:57:55.982826 IP (tos 0x0, ttl 5, id 3966, offset 0, flags [DF], proto UDP (17), length 1498)
10.0.0.1.42193 > 224.1.1.1.5001: UDP, length 1470
mininet>

```

h2 inscrito no multicast

tcpdump escuta pacotes udp no h2

tcpdump escuta pacotes udp no h3

h1 envia pacotes udp multicast

h3 não recebe nenhum pacote multicast pois não está registrado no grupo multicast

h2 recebe os pacotes conforme esperado

Regra OpenFlow instalada no switch após Join de h2

```

mininet> sh ovs-ofctl dump-flows s1
 cookie=0x0, duration=5.383s, table=0, n_packets=0, n_bytes=0, idle_timeout=30, hard_timeout=60, ip,nw
 dst=224.1.1.1 actions=output:"s1-eth2"
mininet>

```

Interpretação dos campos da regra:

- cookie=0x0** : identificador da regra (não definido pelo componente)
- duration** : tempo em segundos desde que a regra foi instalada
- table=0** : tabela de fluxo 0, única tabela no OpenFlow 1.0
- n_packets, n_bytes** : contador de pacotes e bytes que casaram com a regra
- idle_timeout=30** : regra expira após 30s sem tráfego correspondente
- hard_timeout=60** : regra expira após 60s independentemente do tráfego
- ip,nw_dst=224.1.1.1** : condição de casamento: pacotes IPv4 com destino 224.1.1.1
- actions=output:"s1-eth2"** : ação: encaminhar para a porta s1-eth2 (porta de h2)

A porta de h3 (s1-eth3) está ausente das ações, confirmando que o snooping funciona: apenas h2, que enviou o IGMP Join, recebe o tráfego multicast.

Uso de IA Generativa (GPT)

a) Ferramenta Utilizada

- **Claude Code** (Anthropic), modelo `claude-sonnet-4-6`, via CLI no terminal

b) Objetivo do Uso

- Sugestão e avaliação de funcionalidades candidatas para o projeto
- Geração do esboço de `igmp_snooping.py`
- Geração do script de topologia de demonstração (`topo_igmp_demo.py`)
- Estruturação desta documentação

c) Prompts e Respostas Relevantes

Prompt 1: Escolha da funcionalidade

"I need to implement a new feature for POX. Our initial idea was OpenFlow 1.1 but we want something simpler. Suggest alternatives."

Retorno resumido: sugestão de três opções (IGMP Snooping, TCP SYN flood detector, VLAN-aware forwarding) com justificativas de complexidade e demonstrabilidade. **Aproveitado:** escolha do IGMP Snooping como tema; estrutura da demonstração ausência/presença. **Descartado:** opções alternativas (TCP SYN, VLAN).

Prompt 2: Script de topologia de demonstração

"Create a simple mininet infrastructure to demonstrate the lack of IGMP snooping"

Retorno resumido: geração de `ext/topo_igmp_demo.py` com topologia Mininet de 3 hosts e 1 switch, adição automática de rotas multicast em cada host, escrita de PIDs em `/tmp/igmp_demo_pids` e exibição das instruções de uso no terminal.

```

from mininet.net import Mininet
from mininet.node import OVSSwitch, RemoteController
from mininet.cli import CLI
from mininet.log import setLogLevel

def run():
    setLogLevel('info')

    net = Mininet(switch=OVSSwitch, controller=RemoteController)

    net.addController('c0', ip='127.0.0.1', port=6633)
    s1 = net.addSwitch('s1')
    h1 = net.addHost('h1')
    h2 = net.addHost('h2')
    h3 = net.addHost('h3')

    net.addLink(h1, s1)
    net.addLink(h2, s1)
    net.addLink(h3, s1)

    net.start()

    for host in [h1, h2, h3]:
        host.cmd('route add -net 224.0.0.0 netmask 240.0.0.0 dev %s-eth0' % host.name)

    with open('/tmp/igmp_demo_pids', 'w') as f:
        f.write(f"{net['h2'].pid}\n{net['h3'].pid}\n")

    CLI(net)
    net.stop()

if __name__ == '__main__':
    run()

```

Aproveitado: estrutura completa do script; adição das rotas multicast (necessária após erro "Network is unreachable" no teste real).

Descartado: escrita dos PIDs em `/tmp/igmp_demo_pids` (era para integração com script tmux removido posteriormente).

Prompt 3: Implementação do componente igmp_snooping

"Sketch the igmp_snooping.py module"

Retorno resumido: geração de `pox/forwarding/igmp_snooping.py` com parser manual IGMPv2 via `struct.unpack`, classe `IGMPSnooping` com handlers para `ConnectionUp` e `PacketIn`, lógica de Join/Leave, instalação de regras OpenFlow e flood de fallback. **Aproveitado:** estrutura completa do componente; parser manual `parse_igmp()`; lógica de `_install_multicast_flow` e `_install_drop_flow`. **Descartado:** nada de relevância; código revisado e validado integralmente pelo grupo.

d) Rastreabilidade no Código

Arquivo	Influência da IA
<code>pox/forwarding/igmp_snooping.py</code>	esboço gerado pela IA; revisado e validado pelo grupo
<code>ext/topo_igmp_demo.py</code>	gerado pela IA; rotas multicast adicionadas após teste real
<code>docs/relatorio.md</code>	estrutura gerada pela IA; conteúdo técnico preenchido pelo grupo

e) Validação Técnica

- **Conferência com RFC 2236 (IGMPv2):** os offsets e tamanhos dos campos `type` (byte 0), `max_resp_time` (byte 1), `checksum` (bytes 2-3) e `group address` (bytes 4-7) foram conferidos contra a especificação. O formato `struct.unpack("!BBH4s", ...)` foi verificado campo a campo.
- **Validação via tcpdump:** captura real com `tcpdump -v` confirmou que os pacotes IGMP enviados pelo iperf contêm `type=0x16` (Membership Report) no momento do join, e que o campo group address corresponde a `224.1.1.1`.
- **Validação via ovs-ofctl:** o comando `ovs-ofctl dump-flows s1` confirmou que a regra instalada pelo componente contém `nw_dst=224.1.1.1` e `actions=output:"s1-eth2"` (porta de h2), sem incluir a porta de h3.
- **Teste de ausência vs presença:** execução com `l2_learning` mostrou h3 recebendo pacotes (flood); execução com `igmp_snooping` mostrou `0 packets captured` em h3 e recepção normal em h2. Evidências registradas nas seções 5.